Advanced Terraform on AWS Lab 4
Remote State & Multiple Environments

Expected Duration

This lab should take 50-60 minutes to complete

Try to avoid skipping straight to actionable lab steps as the logic of what you are trying to achieve may be lost and the lab learning value will therefore be reduced.

Teaching Points

This lab expands your Module 3 configuration by introducing remote state and multi-environment structure. Local state files work only for a single user on a single machine. Remote state enables collaborative, safe Terraform usage for teams.

- Build an S3 bucket to hold Terraform state.
- Migrate your existing Module 3 state into remote storage.
- Create multiple environment configurations (DEV, TEST, PROD).
- Learn how Terraform uses backend keys to isolate environments.
- Understand why backend values must be literal and cannot use variables.

Before You Begin

- Work only inside the lab4\guided folder.
- Use the lab4\original folder for rollback if required.
- Always specify --var-file when deploying environment-specific configurations.
- When switching backend keys, always run terraform init -reconfigure.

- The remote backend created in this lab will be reused throughout the remainder of the course.

Scenario

In Module 3 you successfully deployed a shared networking environment using Terraform. The deployment works correctly, but it currently stores its state locally on your machine.

This approach works for a single administrator, but creates challenges when Terraform is used by a team:

- Other administrators cannot safely manage the same infrastructure.
- Local state files can be lost, corrupted, or accidentally modified.
- There is no simple way to separate DEV, TEST, and PROD deployments whilst reusing the same Terraform code.

In this lab you will migrate your existing Module 3 deployment from local state to a shared remote backend stored in Amazon S3. You will then extend the solution to support multiple environments using separate state files and environment-specific configuration files.

By the end of the lab you will have:

- A shared remote backend.
- Environment-specific state files.
- Separate DEV, TEST, and PROD deployments.
- A reusable multi-environment Terraform structure.

Phase 1 - Baseline: Run Starter Code Locally

1. Navigate to the lab working directory...

   ```
   cd c:\qatfadvaws\lab4\guided
   ```

2. A script has been provided to surface details of the statefile being used. Unblock this script and run it...

   ```
   Unblock-File .\Print-Statefile.ps1
   .\Print-Statefile.ps1
   ```

   Review the output; there is no current statefile

3. The codebase provided for this lab is carried forward from Lab 3. Initialize and deploy the 22 resources and then re-run the state script;

   ```
   terraform init
   terraform apply --auto-approve
   .\Print-Statefile.ps1
   ```

   Confirm that terraform.tfstate now exists locally and that the state checker reports LOCAL STATE.

Phase 2 - Create the Remote State Backend

Creating a remote state backend to store Terraform state, using Terraform itself, presents a chicken-and-egg challenge.

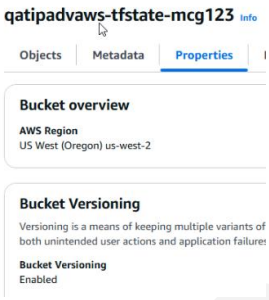
Manual configuration or terraform bootstrap configurations are often used to deal with this issue; a bootstrap deployment creates a local statefile that can either be retained or destroyed whilst leaving the deployed state backend intact.

4. Navigate to **artifacts\lz-state-backend-bootstrap**. Update **terraform.tfvars** with a globally unique bucket name and your preferred AWS region (leave as us-west-2 when running labs on QA Platform).

5. Initialize and apply;

```
cd c:\qatfadvaws\artifacts\lz-state-backend-bootstrap
terraform init
terraform apply --auto-approve
```

Use the AWS Console to verify that the S3 bucket has been created and that Versioning is enabled.



6. At this point you could delete the entire bootstrap configuration whilst leaving its deployment, the S3 bucket, intact. For this lab, leave the configuration as-is in case you wish to use terraform to destroy/recreate the bucket at any point.

Phase 3 - Configure and Migrate to Remote State

Terraform is currently storing state locally. The next step migrates the existing state into the S3 backend without recreating any infrastructure.

7. Navigate back to **lab4/guided** and add the following backend block to the **terraform** block in **providers.tf:**

```
backend "s3" {
  bucket       = "your-bucket-name"
  key          = "default.tfstate"
  region       = "your-region"
  encrypt      = true
  use_lockfile = true
}
```

Backend values must be literal values and cannot use variables.

Update the bucket name to reflect your S3 bucket name and the region to reflect your S3 bucket region (us-west-2)

8. Re-initialize terraform to apply the new configuration and migrate the existing local statefile to the S3 bucket...

```
cd c:\qatfadvaws\lab4\guided
terraform init -migrate-state
```

Answer **yes** when prompted and review the state file status...

```
.\Print-Statefile.ps1
```

Verify that the checker now reports REMOTE STATE - AWS S3 and that **default.tfstate** exists in the S3 bucket.

9. Delete deployed resources, tracked in default.tfstate, before moving onto the next phase where you will create environment specific state files.

```
terraform destroy --auto-approve
```

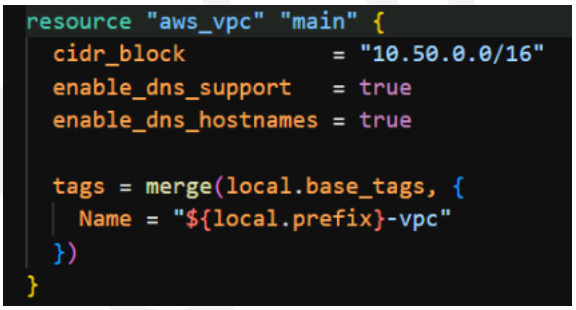
Phase 4 - Introduce Multiple Environments

Ensure you have deleted all deployed resources before proceeding, other than the remote statefile S3 bucket.

Before creating multiple environments, we need to check that the configuration itself is environment-ready. Separate state files do not automatically make the infrastructure design safe. In this configuration, the VPC CIDR is currently hard-coded. If DEV, TEST, and PROD all use the same VPC range, the environments may deploy successfully, but they will be difficult to connect or integrate later because of overlapping IP ranges.

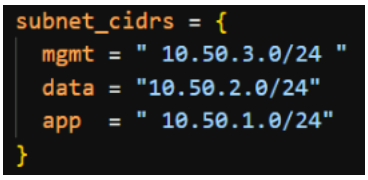
Terraform state files isolate the management of infrastructure, but they do not automatically isolate network design.

Review the current main.tf ...



```
resource "aws_vpc" "main" {
  cidr_block           = "10.50.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = merge(local.base_tags, {
    Name = "${local.prefix}-vpc"
  })
}
```

Review the current terraform.tfvars ...



```
subnet_cidrs = {
  mgmt = " 10.50.3.0/24 "
  data = "10.50.2.0/24"
  app  = " 10.50.1.0/24"
}
```

Using these static settings across multiple environments would result in duplicate networking address ranges being used. Whilst AWS allows multiple VPCs to use identical CIDR ranges, overlapping address spaces can create significant challenges

when environments need to communicate with each other or with on-premises networks.

For this reason, each environment should use a unique VPC CIDR range and matching subnet ranges.

10. Define a new **vpc_cidr** variable in **variables.tf** …

```
variable "vpc_cidr" {
  type        = string
  description = "CIDR block used by the VPC. Must be a valid /16 CIDR
block."

  validation {
    condition = (
      can(cidrhost(trimspace(var.vpc_cidr), 0)) &&
      tonumber(split("/", trimspace(var.vpc_cidr))[1]) == 16
    )

    error_message = "The VPC CIDR must be a valid /16 CIDR block, for
example 10.50.0.0/16."
  }
}
```

11. Update **main.tf** to use this new variable…

```
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = merge(local.base_tags, {
    Name = "${local.prefix}-vpc"
  })
}
```

Separate state files isolate infrastructure management. Environment-specific variable files allow each environment to have its own configuration.

12. Update **terraform.tfvars** with a value for the new variable…

```
vpc_cidr = "10.50.0.0/16"
```

13. Rename **terraform.tfvars** to **dev.tfvars**.
14. Duplicate **dev.tfvars** as **test.tfvars** and **prod.tfvars.**
15. Leaving **dev.tfvars** as-is, update the environment-specific values in the other tfvars files…

    test.tfvars: env = **"TEST"**

    prod.tfvars: env = "**PROD**"

16. Leaving **dev.tfvars** as-is, update the vpc_cidr and subnet_cidrs values in the other tfvars files…

    test.tfvars:

```
subnet_cidrs = {
 mgmt = " 10.51.3.0/24 "
```

```
 data = "10.51.2.0/24"
 app  = " 10.51.1.0/24"
}
vpc_cidr = "10.51.0.0/16"
```

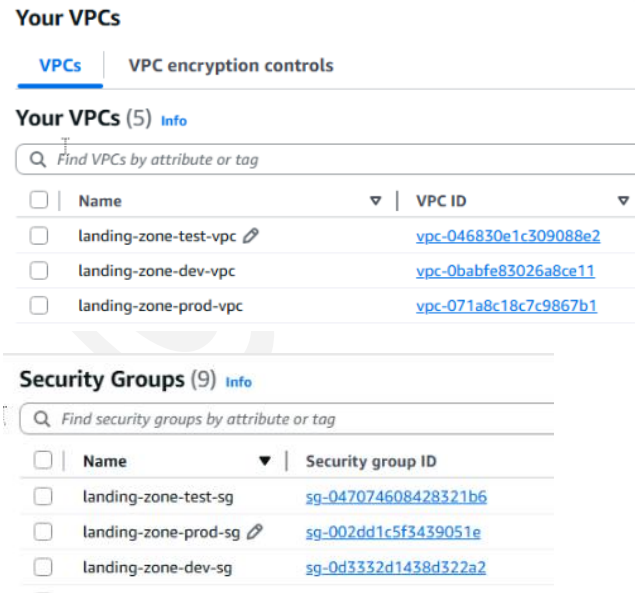
prod.tfvars:

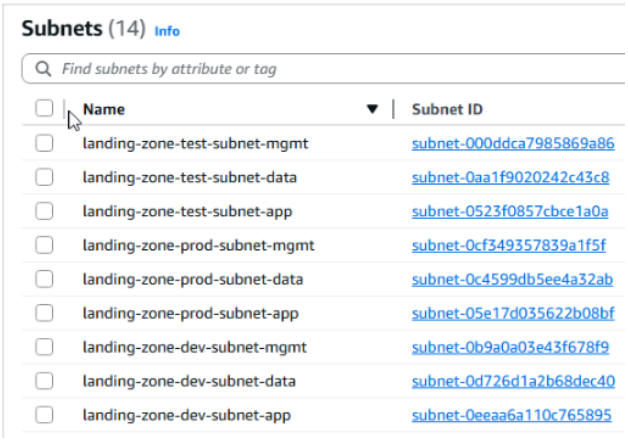
```
subnet_cidrs = {
 mgmt = " 10.52.3.0/24 "
 data = "10.52.2.0/24"
 app  = " 10.52.1.0/24"
}
vpc_cidr = "10.52.0.0/16"
```

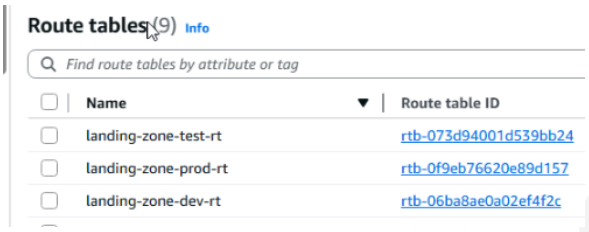
17. Deploy each environment using a separate backend key and verify the backend state files changes;

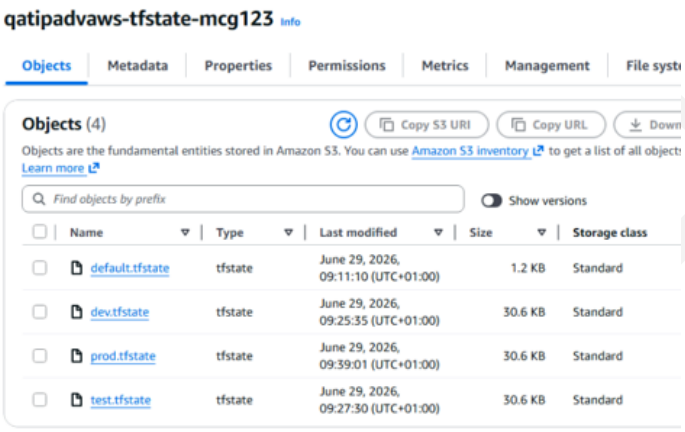
```
terraform init -reconfigure -backend-config="key=dev.tfstate"
terraform apply --var-file=dev.tfvars --auto-approve
.\Print-Statefile.ps1


terraform init -reconfigure -backend-config="key=test.tfstate"
terraform apply --var-file=test.tfvars --auto-approve
.\Print-Statefile.ps1


terraform init -reconfigure -backend-config="key=prod.tfstate"
terraform apply --var-file=prod.tfvars --auto-approve
.\Print-Statefile.ps1
```

18. Switch to the AWS Console and verify the creation of the three environment resources and the separate state files in the S3 bucket, used to track each..









Phase 5 - Cleanup

Destroy the DEV, TEST and PROD environments. Do not destroy the backend bucket as it will be reused in later labs.

```
terraform init -reconfigure -backend-config="key=dev.tfstate"
terraform destroy --var-file=dev.tfvars --auto-approve

terraform init -reconfigure -backend-config="key=test.tfstate"
terraform destroy --var-file=test.tfvars --auto-approve

terraform init -reconfigure -backend-config="key=prod.tfstate"
terraform destroy --var-file=prod.tfvars --auto-approve
```

***** Congratulations, you have completed this lab *****

Copyright